

| Area                | Your Choice                                          |
|---------------------|------------------------------------------------------|
| Application         | Single web application                               |
| Stack               | Java + Maven + TestNG                                |
| Browser             | Configurable browser                                 |
| Test data sources   | Excel + CSV + JSON + Database                        |
| Data mapping        | TestNG @DataProvider                                 |
| Environment config  | Single configuration file                            |
| Config format       | .properties                                          |
| Page Object Model   | Page Objects + reusable Components                   |
| WebDriver lifecycle | One driver for entire suite                          |
| Parallel execution  | Configurable                                         |
| Test selection      | TestNG Groups                                        |
| Locators            | Inside Page/Component classes                        |
| Synchronization     | Explicit waits                                       |
| Reporting           | TestNG default reports + Java logging                |
| Failure evidence    | Screenshot + TestNG report                           |
| Retry               | Automatic retry                                      |
| Data readers        | Separate reader per source                           |
| Data organization   | Centralized test-data file per format                |
| Sensitive data      | Separate credentials/configuration                   |
| Authentication      | Login before every test                              |
| Driver management   | Selenium Manager                                     |
| Architecture        | Test → Business/Workflow → Page/Component → Selenium |
| Exception handling  | Custom exceptions + centralized handling             |
| CI/CD               | Jenkins                                              |
| Source control      | Git + GitHub                                         |
| Dependency strategy | Latest compatible versions                           |
| Browser mode        | Configurable headed/headless                         |

## 1. One architectural point I'll explicitly address

There is a potential tension between:

- **One WebDriver for the entire suite**
- **Configurable parallel execution**
- **Login before every test**

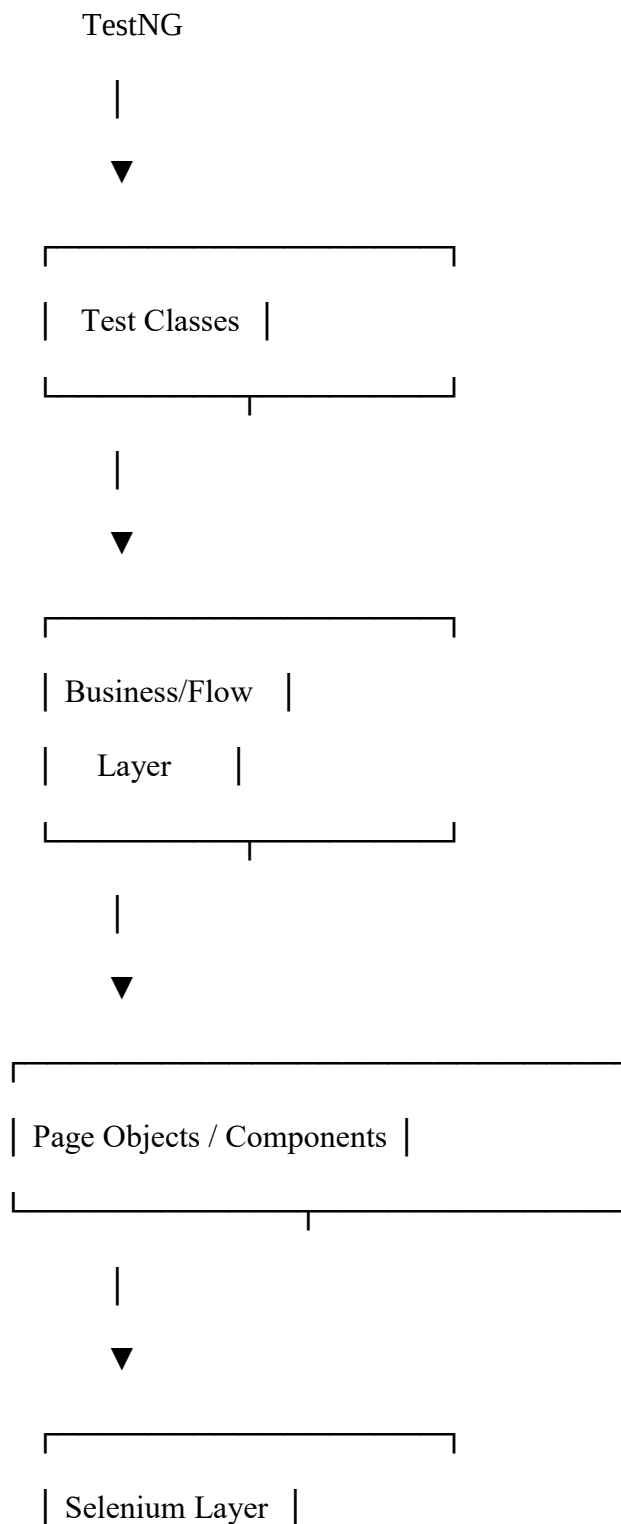
A single WebDriver instance cannot safely support true parallel test execution because Selenium WebDriver is not inherently thread-safe.

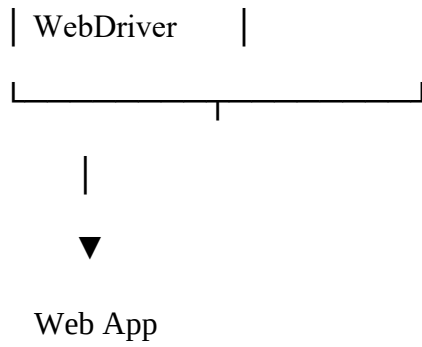
Therefore, in the architecture explanation I'll distinguish **your selected default lifecycle** from the **parallel execution architecture**, rather than silently changing your requirement.

I will also preserve your choice of **latest compatible dependencies**, while explaining the reproducibility trade-off and how Maven can manage this safely.

## 2. Proposed Architecture

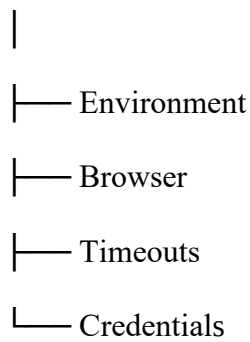
The framework will follow this conceptual flow:



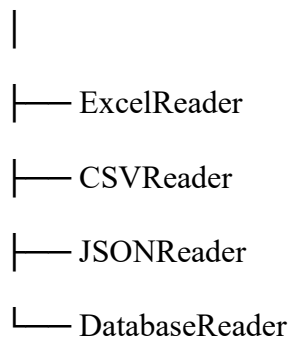


Supporting services will sit alongside this flow:

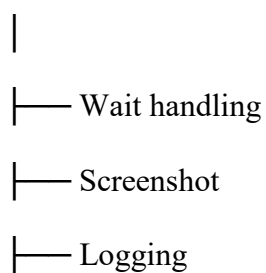
#### Configuration



#### Test Data



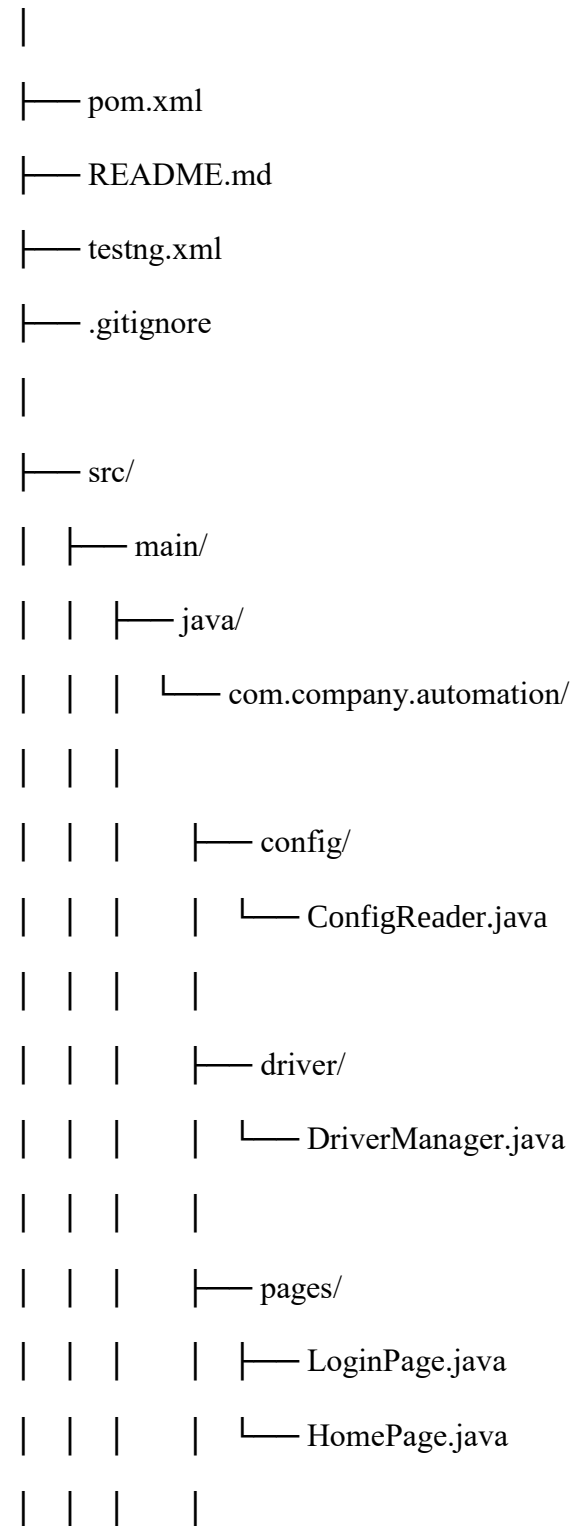
#### Framework Utilities



- └─ Retry
- └─ Exception handling

### 3. Recommended Project Structure

selenium-automation-framework/



```
| | | | └── components/
| | | | | └── HeaderComponent.java
| | | | | └── MenuComponent.java
| | | | |
| | | | └── workflows/
| | | | | └── LoginWorkflow.java
| | | | | └── UserWorkflow.java
| | | | |
| | | | └── data/
| | | | | └── ExcelReader.java
| | | | | └── CSVReader.java
| | | | | └── JSONReader.java
| | | | | └── DatabaseReader.java
| | | | |
| | | | └── exceptions/
| | | | | └── ConfigurationException.java
| | | | | └── DataReaderException.java
| | | | |
| | | | └── utils/
| | | | | └── ScreenshotUtils.java
| | | | | └── WaitUtils.java
| | | |
| | └── resources/
| | └── config/
| | └── └── config.properties
```

```

| | |
| | └─ testdata/
| |   └─ testdata.xlsx
| |   └─ testdata.csv
| |   └─ testdata.json
| |
| └─ test/
|   └─ java/
|     └─ com.company.automation.tests/
|       └─ LoginTest.java
|       └─ UserTest.java
|
| └─ resources/
|
└─ reports/

```

#### 4. Core Design Decisions

##### Page + Component Model

Rather than putting everything into traditional page classes:

LoginPage

HomePage

PaymentPage

we'll allow reusable UI components:

HeaderComponent

MenuComponent

TableComponent

ModalComponent

DatePickerComponent

This becomes particularly valuable when the same UI element appears on multiple pages.

## Workflow Layer

Your choice of a workflow layer is particularly useful for complex applications.

For example:

```
loginWorkflow.login(username, password);
```

instead of making every test understand:

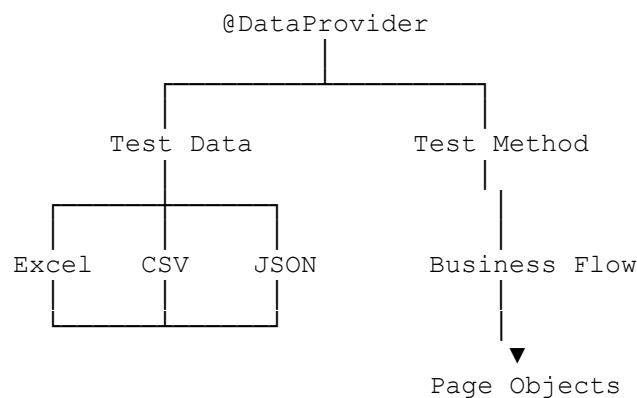
```
loginPage.enterUsername();  
loginPage.enterPassword();  
loginPage.clickLogin();  
homePage.verifyHomePage();
```

The test remains business-focused.

---

## 5. Data-Driven Architecture

Your selected approach allows:



Database data can be added through the same overall data-access concept.

---

## 6. Important Selenium Design

You selected **explicit waits only**.

Therefore the framework will avoid:

```
Thread.sleep(5000);
```

and avoid using implicit waits.

Instead:

```
WebDriverWait wait =  
    new WebDriverWait(driver, Duration.ofSeconds(10));  
  
wait.until(ExpectedConditions.elementToBeClickable(element));
```

This gives condition-based synchronization and avoids unnecessary waiting.

---

## 7. WebDriver Strategy

Your selected default is:

### **One WebDriver for the entire suite**

We'll implement that as the initial lifecycle.

However, for the configurable parallel mode, the architecture should support:

```
Sequential execution  
  |  
  ▼  
Single WebDriver
```

and when parallel execution is enabled:

```
Test 1 → Driver 1  
Test 2 → Driver 2  
Test 3 → Driver 3
```

This is where a `ThreadLocal<WebDriver>` based implementation becomes appropriate.

That allows the framework to remain scalable without requiring a redesign later.

---

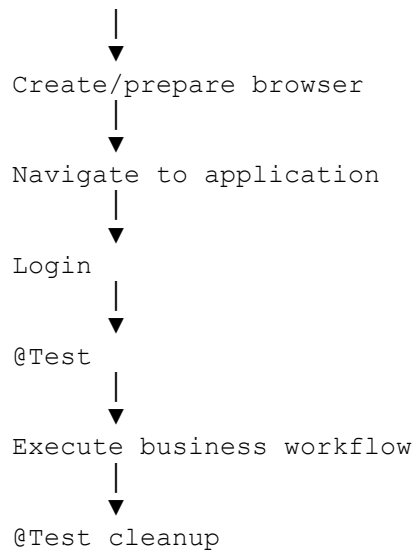
## 8. Authentication

You selected:

Login before every test.

The resulting lifecycle will conceptually be:

```
@BeforeMethod
```

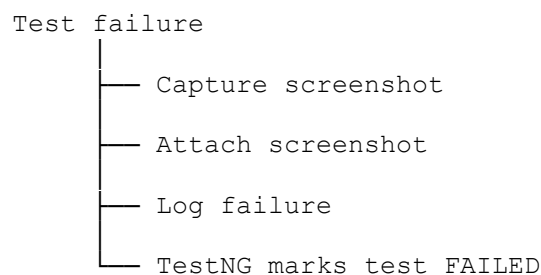


This gives tests strong isolation.

---

## 9. Failure Handling

On failure:



Your automatic retry mechanism will then provide another execution attempt.

One caution: retries should be treated as a **diagnostic mechanism**, not a way to hide genuine application defects. We'll structure the implementation accordingly.

---

## 10. Implementation Plan

I'll build the framework in this order:

### Step 1

Create Maven project and `pom.xml`.

### Step 2

Create configuration management using:

```
config.properties  
ConfigReader.java
```

### **Step 3**

Implement browser/driver management using Selenium Manager.

### **Step 4**

Implement Base Selenium functionality and explicit synchronization.

### **Step 5**

Implement Page Objects.

### **Step 6**

Implement reusable Components.

### **Step 7**

Implement Business/Workflow layer.

### **Step 8**

Implement Excel, CSV, JSON and Database readers.

### **Step 9**

Implement TestNG @DataProvider.

### **Step 10**

Implement test classes and TestNG Groups.

### **Step 11**

Implement login-before-each-test lifecycle.

### **Step 12**

Implement screenshots and TestNG reporting.

### **Step 13**

Implement retry mechanism.

## Step 14

Implement custom exceptions and centralized error handling.

## Step 15

Implement configurable browser/headless execution.

## Step 16

Implement configurable parallel execution.

## Step 17

Create TestNG suite configuration.

## Step 18

Create Jenkins execution configuration.

## Step 19

Add Git/GitHub-friendly project configuration.

## Step 20

Create a complete end-to-end sample test demonstrating the framework.

The implementation will use **well-commented Java code**, and after each major component I'll explain **why it exists, how it works, and what alternatives could have been used**.